ONLINE HEALTH CARE MANAGEMENT SYSTEM

Upgraded Complete Java GUI + MySQL + JDBC Project

Academic Project Code Document

Technology: Java Swing | JDBC | MySQL | Maven

This PDF contains the complete source code and database script of the upgraded project. Extracted source files are presented sequentially by package and file.

1. Project Features

Role-based login, Admin/Doctor/Patient dashboards, doctor and patient management, appointment booking and status workflow, medical records, JDBC database connectivity, DAO architecture, OOP, collections, generics, exception handling, multithreading and synchronization.

2. Project Structure

```
Online-Healthcare-Management-System-Strong/
├── database/healthcare.sql
├── src/com/healthcare/Main.java
├── src/com/healthcare/model/
├── src/com/healthcare/dao/
├── src/com/healthcare/gui/
├── src/com/healthcare/service/
├── src/com/healthcare/exception/
├── src/com/healthcare/util/
├── screenshots/
├── README.md
└── pom.xml
```

database/healthcare.sql

```
CREATE DATABASE IF NOT EXISTS healthcare;
USE healthcare;

DROP TABLE IF EXISTS medical_records;
DROP TABLE IF EXISTS appointments;
DROP TABLE IF EXISTS patients;
DROP TABLE IF EXISTS doctors;
DROP TABLE IF EXISTS users;

CREATE TABLE users(
 user_id INT PRIMARY KEY AUTO_INCREMENT,
 username VARCHAR(50) UNIQUE NOT NULL,
 password VARCHAR(255) NOT NULL,
 role ENUM('ADMIN','DOCTOR','PATIENT') NOT NULL,
 active BOOLEAN DEFAULT TRUE
);

CREATE TABLE patients(
 patient_id INT PRIMARY KEY AUTO_INCREMENT,
 user_id INT UNIQUE,
 name VARCHAR(100) NOT NULL,
 age INT NOT NULL,
 gender VARCHAR(20),
 phone VARCHAR(20),
 email VARCHAR(100),
 address VARCHAR(255),
 FOREIGN KEY(user_id) REFERENCES users(user_id) ON DELETE SET NULL
);

CREATE TABLE doctors(
 doctor_id INT PRIMARY KEY AUTO_INCREMENT,
 user_id INT UNIQUE,
 name VARCHAR(100) NOT NULL,
 specialization VARCHAR(100) NOT NULL,
 phone VARCHAR(20),
 email VARCHAR(100),
 available_time VARCHAR(100),
 consultation_fee DECIMAL(10,2) DEFAULT 500,
 FOREIGN KEY(user_id) REFERENCES users(user_id) ON DELETE SET NULL
);

CREATE TABLE appointments(
 appointment_id INT PRIMARY KEY AUTO_INCREMENT,
 patient_id INT NOT NULL,
 doctor_id INT NOT NULL,
 appointment_date DATE NOT NULL,
 appointment_time TIME NOT NULL,
 reason VARCHAR(255),
 status ENUM('Pending','Approved','Rejected','Completed') DEFAULT 'Pending',
 FOREIGN KEY(patient_id) REFERENCES patients(patient_id) ON DELETE CASCADE,
 FOREIGN KEY(doctor_id) REFERENCES doctors(doctor_id) ON DELETE CASCADE
);

CREATE TABLE medical_records(
 record_id INT PRIMARY KEY AUTO_INCREMENT,
 patient_id INT NOT NULL,
 doctor_id INT NOT NULL,
 diagnosis VARCHAR(255) NOT NULL,
 prescription VARCHAR(500),
 notes VARCHAR(500),
 record_date DATE NOT NULL,
 FOREIGN KEY(patient_id) REFERENCES patients(patient_id) ON DELETE CASCADE,
 FOREIGN KEY(doctor_id) REFERENCES doctors(doctor_id) ON DELETE CASCADE
);

INSERT INTO users(username,password,role) VALUES
('admin','admin123','ADMIN'),
('doctor1','doctor123','DOCTOR'),
('patient1','patient123','PATIENT');

INSERT INTO doctors(user_id,name,specialization,phone,email,available_time,consultation_fee)
VALUES(2,'Dr. Sharma','Cardiologist','9876543210','sharma@healthcare.com','10:00 AM - 2:00 PM',800);

INSERT INTO patients(user_id,name,age,gender,phone,email,address)
VALUES(3,'Rahul Kumar',21,'Male','9876501234','rahul@gmail.com','Ghaziabad');
```

src/com/healthcare/Main.java

```
 1: package com.healthcare;
 2:
 3: import com.healthcare.gui.LoginFrame;
 4: import com.healthcare.service.NotificationService;
 5: import javax.swing.*;
 6:
 7: public class Main {
 8:     public static void main(String[] args) {
 9:         NotificationService service = new NotificationService();
10:         service.start();
11:         SwingUtilities.invokeLater(() -> new LoginFrame().setVisible(true));
12:     }
13: }
```

src/com/healthcare/util/DBConnection.java

```
 1: package com.healthcare.util;
 2:
 3: import java.sql.Connection;
 4: import java.sql.DriverManager;
 5: import java.sql.SQLException;
 6:
 7: public final class DBConnection {
 8:     private static final String URL="jdbc:mysql://localhost:3306/healthcare?useSSL=false&serverTimezone=UTC";
 9:     private static final String USER="root";
10:     private static final String PASSWORD="root";
11:
12:     private DBConnection(){}
13:
14:     public static Connection getConnection() throws SQLException {
15:         return DriverManager.getConnection(URL,USER,PASSWORD);
16:     }
17: }
```

src/com/healthcare/exception/ValidationException.java

```
1: package com.healthcare.exception;
2:
3: public class ValidationException extends Exception {
4:     public ValidationException(String message){ super(message); }
5: }
```

src/com/healthcare/dao/CRUDOperations.java

```
1: package com.healthcare.dao;
2:
3: public interface CRUDOperations<T> {
4:     void add(T obj) throws Exception;
5:     void update(T obj) throws Exception;
6:     void delete(int id) throws Exception;
7: }
```

src/com/healthcare/model/User.java

```
 1: package com.healthcare.model;
 2:
 3: public class User {
 4:     private int id;
 5:     private String username,password,role;
 6:
 7:     public User(){}
 8:     public User(int id,String username,String password,String role){
 9:         this.id=id;this.username=username;this.password=password;this.role=role;
10:     }
11:     public int getId(){return id;}
12:     public String getUsername(){return username;}
13:     public String getPassword(){return password;}
14:     public String getRole(){return role;}
15:     public void showDashboard(){System.out.println("User Dashboard");}
16: }
```

src/com/healthcare/model/Patient.java

```
 1: package com.healthcare.model;
 2:
 3: public class Patient extends User {
 4:     private int patientId,age;
 5:     private String name,gender,phone,email,address;
 6:
 7:     public Patient(){}
 8:     public Patient(int id,String name,int age,String gender,String phone,String email,String address){
 9:         this.patientId=id;this.name=name;this.age=age;this.gender=gender;
10:          this.phone=phone;this.email=email;this.address=address;
11:     }
12:     public int getPatientId(){return patientId;}
13:     public String getName(){return name;}
14:     public int getAge(){return age;}
15:     public String getGender(){return gender;}
16:     public String getPhone(){return phone;}
17:     public String getEmail(){return email;}
18:     public String getAddress(){return address;}
19:     @Override public void showDashboard(){System.out.println("Patient Dashboard");}
20: }
```

src/com/healthcare/model/Doctor.java

```
 1: package com.healthcare.model;
 2:
 3: public class Doctor extends User {
 4:     private int doctorId;
 5:     private String name,specialization,phone,email,availableTime;
 6:     private double fee;
 7:
 8:     public Doctor(){}
 9:     public Doctor(int id,String name,String specialization,String phone,String email,String time,double fee){
10:         this.doctorId=id;this.name=name;this.specialization=specialization;this.phone=phone;
11:         this.email=email;this.availableTime=time;this.fee=fee;
12:     }
13:     public int getDoctorId(){return doctorId;}
14:     public String getName(){return name;}
15:     public String getSpecialization(){return specialization;}
16:     public String getPhone(){return phone;}
17:     public String getEmail(){return email;}
18:     public String getAvailableTime(){return availableTime;}
19:     public double getFee(){return fee;}
20:     @Override public void showDashboard(){System.out.println("Doctor Dashboard");}
21:     @Override public String toString(){return doctorId+" - "+name+" ("+specialization+")";}
22: }
```

src/com/healthcare/model/Appointment.java

```
 1: package com.healthcare.model;
 2:
 3: import java.sql.Date;
 4: import java.sql.Time;
 5:
 6: public class Appointment {
 7:     private int id,patientId,doctorId;
 8:     private Date date;
 9:     private Time time;
10:     private String reason,status;
11:
12:     public Appointment(int patientId,int doctorId,Date date,Time time,String reason){
13:         this.patientId=patientId;this.doctorId=doctorId;this.date=date;this.time=time;this.reason=reason;
14:         this.status="Pending";
15:     }
16:     public int getId(){return id;}
17:     public int getPatientId(){return patientId;}
18:     public int getDoctorId(){return doctorId;}
19:     public Date getDate(){return date;}
20:     public Time getTime(){return time;}
21:     public String getReason(){return reason;}
22:     public String getStatus(){return status;}
23: }
```

src/com/healthcare/model/MedicalRecord.java

```
 1: package com.healthcare.model;
 2:
 3: import java.sql.Date;
 4:
 5: public class MedicalRecord {
 6:     private int patientId,doctorId;
 7:     private String diagnosis,prescription,notes;
 8:     private Date date;
 9:
10:     public MedicalRecord(int p,int d,String diagnosis,String prescription,String notes,Date date){
11:         this.patientId=p;this.doctorId=d;this.diagnosis=diagnosis;this.prescription=prescription;
12:         this.notes=notes;this.date=date;
13:     }
14:     public int getPatientId(){return patientId;}
15:     public int getDoctorId(){return doctorId;}
16:     public String getDiagnosis(){return diagnosis;}
17:     public String getPrescription(){return prescription;}
18:     public String getNotes(){return notes;}
19:     public Date getDate(){return date;}
20: }
```

src/com/healthcare/dao/UserDAO.java

```
 1: package com.healthcare.dao;
 2:
 3: import com.healthcare.model.User;
 4: import com.healthcare.util.DBConnection;
 5: import java.sql.*;
 6:
 7: public class UserDAO {
 8:     public User login(String username,String password) throws SQLException {
 9:         String q="SELECT * FROM users WHERE username=? AND password=? AND active=TRUE";
10:         try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement(q)){
11:             p.setString(1,username);p.setString(2,password);
12:             ResultSet r=p.executeQuery();
13:             if(r.next()) return new User(r.getInt("user_id"),r.getString("username"),r.getString("password"),r.getString("role"));
14:         }
15:         return null;
16:     }
17: }
```

src/com/healthcare/dao/DoctorDAO.java

```
 1: package com.healthcare.dao;
 2:
 3: import com.healthcare.model.Doctor;
 4: import com.healthcare.util.DBConnection;
 5: import java.sql.*;
 6: import java.util.*;
 7:
 8: public class DoctorDAO implements CRUDOperations<Doctor> {
 9:     public void add(Doctor d)throws Exception{
10:         String q="INSERT INTO doctors(name,specialization,phone,email,available_time,consultation_fee) VALUES(?,?,?,?,?,?)";
11:         try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement(q)){
12:             p.setString(1,d.getName());p.setString(2,d.getSpecialization());p.setString(3,d.getPhone());
13:             p.setString(4,d.getEmail());p.setString(5,d.getAvailableTime());p.setDouble(6,d.getFee());p.executeUpdate();
14:         }
15:     }
16:     public void update(Doctor d)throws Exception{}
17:     public void delete(int id)throws Exception{
18:         try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement("DELETE FROM doctors WHERE doctor_id=?")){
19:             p.setInt(1,id);p.executeUpdate();
20:         }
21:     }
22:     public List<Doctor> all()throws Exception{
23:         List<Doctor> x=new ArrayList<>();
24:         try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement("SELECT * FROM doctors");ResultSet r=p.executeQuery()){
25:             while(r.next())x.add(new Doctor(r.getInt("doctor_id"),r.getString("name"),r.getString("specialization"),
26:             r.getString("phone"),r.getString("email"),r.getString("available_time"),r.getDouble("consultation_fee")));
27:         }return x;
28:     }
29: }
```

src/com/healthcare/dao/PatientDAO.java

```
 1: package com.healthcare.dao;
 2:
 3: import com.healthcare.model.Patient;
 4: import com.healthcare.util.DBConnection;
 5: import java.sql.*;
 6: import java.util.*;
 7:
 8: public class PatientDAO implements CRUDOperations<Patient> {
 9:     public void add(Patient p)throws Exception{
10:         String q="INSERT INTO patients(name,age,gender,phone,email,address) VALUES(?,?,?,?,?,?)";
11:         try(Connection c=DBConnection.getConnection();PreparedStatement s=c.prepareStatement(q)){
12:             s.setString(1,p.getName());s.setInt(2,p.getAge());s.setString(3,p.getGender());
13:             s.setString(4,p.getPhone());s.setString(5,p.getEmail());s.setString(6,p.getAddress());s.executeUpdate();
14:         }
15:     }
16:     public void update(Patient p)throws Exception{}
17:     public void delete(int id)throws Exception{
18:         try(Connection c=DBConnection.getConnection();PreparedStatement s=c.prepareStatement("DELETE FROM patients WHERE patient_id=?")){
19:             s.setInt(1,id);s.executeUpdate();
20:         }
21:     }
22:     public List<Patient> all()throws Exception{
23:         List<Patient>x=new ArrayList<>();
24:         try(Connection c=DBConnection.getConnection();PreparedStatement s=c.prepareStatement("SELECT * FROM patients");ResultSet r=s.executeQuery()){
25:             while(r.next())x.add(new Patient(r.getInt("patient_id"),r.getString("name"),r.getInt("age"),
26:             r.getString("gender"),r.getString("phone"),r.getString("email"),r.getString("address")));
27:         }return x;
28:     }
29:     public int findIdByUser(int uid)throws Exception{
30:         try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement("SELECT patient_id FROM patients WHERE user_id=?")){
31:             p.setInt(1,uid);ResultSet r=p.executeQuery();if(r.next())return r.getInt(1);
32:         }throw new Exception("Patient profile not found.");
33:     }
34: }
```

src/com/healthcare/dao/AppointmentDAO.java

```
 1: package com.healthcare.dao;
 2:
 3: import com.healthcare.model.Appointment;
 4: import com.healthcare.util.DBConnection;
 5: import java.sql.*;
 6: import java.util.*;
 7:
 8: public class AppointmentDAO implements CRUDOperations<Appointment>{
 9:     public void add(Appointment a)throws Exception{
10:         String q="INSERT INTO appointments(patient_id,doctor_id,appointment_date,appointment_time,reason) VALUES(?,?,?,?,?)";
11:         try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement(q)){
12:             p.setInt(1,a.getPatientId());p.setInt(2,a.getDoctorId());p.setDate(3,a.getDate());p.setTime(4,a.getTime());
13:             p.setString(5,a.getReason());p.executeUpdate();
14:         }
15:     }
16:     public void update(Appointment a)throws Exception{}
17:     public void delete(int id)throws Exception{
18:         try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement("DELETE FROM appointments WHERE appointment_id=?")){
19:             p.setInt(1,id);p.executeUpdate();
20:         }
21:     }
22:     public void status(int id,String s)throws Exception{
23:         try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement("UPDATE appointments SET status=? WHERE appointment_id=?")){
24:             p.setString(1,s);p.setInt(2,id);p.executeUpdate();
25:         }
26:     }
27:     public List<String[]> all()throws Exception{
28:         List<String[]>x=new ArrayList<>();
29:         String q="SELECT a.appointment_id,p.name patient,d.name doctor,a.appointment_date,a.appointment_time,a.reason,a.status "+
30:                  "FROM appointments a JOIN patients p ON a.patient_id=p.patient_id JOIN doctors d ON a.doctor_id=d.doctor_id ORDER BY a.appointment_date";
31:         try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement(q);ResultSet r=p.executeQuery()){
32:             while(r.next())x.add(new String[]{""+r.getInt(1),r.getString(2),r.getString(3),""+r.getDate(4),""+r.getTime(5),r.getString(6),r.getString(7)});
33:         }return x;
34:     }
35: }
```

src/com/healthcare/dao/MedicalRecordDAO.java

```
 1: package com.healthcare.dao;
 2:
 3: import com.healthcare.model.MedicalRecord;
 4: import com.healthcare.util.DBConnection;
 5: import java.sql.*;
 6: import java.util.*;
 7:
 8: public class MedicalRecordDAO {
 9:     public void add(MedicalRecord m)throws Exception{
10:         String q="INSERT INTO medical_records(patient_id,doctor_id,diagnosis,prescription,notes,record_date) VALUES(?,?,?,?,?,?)";
11:         try(Connection c=DBConnection.getConnection();PreparedStatement p=c.prepareStatement(q)){
12:             p.setInt(1,m.getPatientId());p.setInt(2,m.getDoctorId());p.setString(3,m.getDiagnosis());
13:             p.setString(4,m.getPrescription());p.setString(5,m.getNotes());p.setDate(6,m.getDate());p.executeUpdate();
14:         }
15:     }
16:     public List<String[]> all()throws Exception{
17:         List<String[]>x=new ArrayList<>();
18:         String q="SELECT m.record_id,p.name,d.name,m.diagnosis,m.prescription,m.notes,m.record_date FROM medical_records m "+
19:                 "JOIN patients p ON m.patient_id=p.patient_id JOIN doctors d ON m.doctor_id=d.doctor_id ORDER BY m.record_date DESC";
20:         try(Connection c=DBConnection.getConnection();PreparedStatement s=c.prepareStatement(q);ResultSet r=s.executeQuery()){
21:             while(r.next())x.add(new String[]{""+r.getInt(1),r.getString(2),r.getString(3),r.getString(4),r.getString(5),r.getString(6),""+r.getDate(7)});
22:         }return x;
23:     }
24: }
```

src/com/healthcare/service/NotificationService.java

```
 1: package com.healthcare.service;
 2:
 3: public class NotificationService extends Thread {
 4:     private boolean running=true;
 5:     public void run(){
 6:         while(running){
 7:             System.out.println("[Background Thread] Appointment reminder service running...");
 8:             try{Thread.sleep(5000);}catch(InterruptedException e){running=false;}
 9:         }
10:     }
11:     public synchronized void stopService(){running=false;interrupt();}
12: }
```

src/com/healthcare/service/AppointmentCounter.java

```
1: package com.healthcare.service;
2:
3: public class AppointmentCounter {
4:     private int count;
5:     public synchronized void increment(){count++;}
6:     public synchronized int getCount(){return count;}
7: }
```

src/com/healthcare/gui/UI.java

```
 1: package com.healthcare.gui;
 2:
 3: import javax.swing.*;
 4: import java.awt.*;
 5:
 6: public final class UI {
 7:     private UI(){}
 8:     public static void style(JFrame f){
 9:         f.getContentPane().setBackground(new Color(245,248,250));
10:         UIManager.put("Button.font",new Font("Arial",Font.BOLD,13));
11:         UIManager.put("Label.font",new Font("Arial",Font.PLAIN,13));
12:     }
13:     public static JButton button(String s){return new JButton(s);}
14:     public static void error(Component c,Exception e){JOptionPane.showMessageDialog(c,e.getMessage(),"Error",JOptionPane.ERROR_MESSAGE);}
15: }
```

src/com/healthcare/gui/LoginFrame.java

```
package com.healthcare.gui;

import com.healthcare.dao.UserDAO;
import com.healthcare.model.User;
import javax.swing.*;
import java.awt.*;

public class LoginFrame extends JFrame {
    JTextField username=new JTextField();
    JPasswordField password=new JPasswordField();
    JComboBox<String> role=new JComboBox<>(new String[]{"ADMIN","DOCTOR","PATIENT"});

    public LoginFrame(){
        setTitle("Online Health Care Management System - Login");
        setSize(500,350);setLocationRelativeTo(null);setDefaultCloseOperation(EXIT_ON_CLOSE);
        UI.style(this);

        JPanel box=new JPanel(new GridLayout(5,2,12,12));
        box.setBorder(BorderFactory.createEmptyBorder(35,55,35,55));
        box.add(new JLabel("Username"));box.add(username);
        box.add(new JLabel("Password"));box.add(password);
        box.add(new JLabel("Role"));box.add(role);
        JButton login=UI.button("LOGIN"),clear=UI.button("CLEAR");
        box.add(login);box.add(clear);
        JLabel head=new JLabel("ONLINE HEALTH CARE MANAGEMENT SYSTEM",SwingConstants.CENTER);
        head.setFont(new Font("Arial",Font.BOLD,18));
        add(head,BorderLayout.NORTH);add(box);
        login.addActionListener(e->login());
        clear.addActionListener(e->{username.setText("");password.setText("");});
    }
    void login(){
        try{
            User u=new UserDAO().login(username.getText().trim(),new String(password.getPassword()));
            if(u==null||!u.getRole().equals(role.getSelectedItem())){JOptionPane.showMessageDialog(this,"Invalid login details.");return;}
            dispose();
            if(u.getRole().equals("ADMIN"))new AdminDashboard().setVisible(true);
            else if(u.getRole().equals("DOCTOR"))new DoctorDashboard(u.getId()).setVisible(true);
            else new PatientDashboard(u.getId()).setVisible(true);
        }catch(Exception e){UI.error(this,e);}
    }
}
```

src/com/healthcare/gui/AdminDashboard.java

```
 1: package com.healthcare.gui;
 2:
 3: import com.healthcare.dao.*;
 4: import com.healthcare.model.*;
 5: import javax.swing.*;
 6: import javax.swing.table.DefaultTableModel;
 7: import java.awt.*;
 8:
 9: public class AdminDashboard extends JFrame {
10:     DefaultTableModel model=new DefaultTableModel();
11:     JTable table=new JTable(model);
12:
13:     public AdminDashboard(){
14:         setTitle("Admin Dashboard");setSize(1050,650);setLocationRelativeTo(null);setDefaultCloseOperation(EXIT_ON_CLOSE);UI.style(this);
15:         JPanel bar=new JPanel();
16:         JButton d=UI.button("Doctors"),p=UI.button("Patients"),a=UI.button("Appointments"),m=UI.button("Medical Records"),addD=UI.button("Add Doctor"),addP=UI.button("
17:         for(JButton b:new JButton[]{d,p,a,m,addD,addP,logout})bar.add(b);
18:         add(bar,BorderLayout.NORTH);add(new JScrollPane(table));
19:         d.addActionListener(e->doctors());p.addActionListener(e->patients());a.addActionListener(e->appointments());m.addActionListener(e->records());
20:         addD.addActionListener(e->addDoctor());addP.addActionListener(e->addPatient());
21:         logout.addActionListener(e->{dispose();new LoginFrame().setVisible(true);});
22:         doctors();
23:     }
24:     void doctors(){try{model.setDataVector(new Object[]{"ID","Name","Specialization","Phone","Email","Available","Fee"},0);for(Doctor x:new DoctorDAO().all())model.addRo
25:     void patients(){try{model.setDataVector(new Object[]{"ID","Name","Age","Gender","Phone","Email","Address"},0);for(Patient x:new PatientDAO().all())model.addRow(new
26:     void appointments(){try{model.setDataVector(new Object[]{"ID","Patient","Doctor","Date","Time","Reason","Status"},0);for(String[] x:new AppointmentDAO().all())model
27:     void records(){try{model.setDataVector(new Object[]{"ID","Patient","Doctor","Diagnosis","Prescription","Notes","Date"},0);for(String[] x:new MedicalRecordDAO().all())m
28:
29:     void addDoctor(){
30:         try{
31:             String n=JOptionPane.showInputDialog(this,"Doctor name:");String s=JOptionPane.showInputDialog(this,"Specialization:");
32:             String ph=JOptionPane.showInputDialog(this,"Phone:");String em=JOptionPane.showInputDialog(this,"Email:");
33:             String t=JOptionPane.showInputDialog(this,"Available time:");String fee=JOptionPane.showInputDialog(this,"Consultation fee:");
34:             if(n==null||s==null)return;
35:             new DoctorDAO().add(new Doctor(0,n,s,ph,em,t,Double.parseDouble(fee)));
36:             doctors();
37:         }catch(Exception e){UI.error(this,e);}
38:     }
39:     void addPatient(){
40:         try{
41:             String n=JOptionPane.showInputDialog(this,"Patient name:");String age=JOptionPane.showInputDialog(this,"Age:");
42:             String g=JOptionPane.showInputDialog(this,"Gender:");String ph=JOptionPane.showInputDialog(this,"Phone:");
43:             String em=JOptionPane.showInputDialog(this,"Email:");String ad=JOptionPane.showInputDialog(this,"Address:");
44:             if(n==null)return;
45:             new PatientDAO().add(new Patient(0,n,Integer.parseInt(age),g,ph,em,ad));patients();
46:         }catch(Exception e){UI.error(this,e);}
47:     }
48: }
```

src/com/healthcare/gui/PatientDashboard.java

```
 1: package com.healthcare.gui;
 2:
 3: import com.healthcare.dao.*;
 4: import com.healthcare.model.*;
 5: import javax.swing.*;
 6: import javax.swing.table.DefaultTableModel;
 7: import java.awt.*;
 8: import java.sql.Date;
 9: import java.sql.Time;
10:
11: public class PatientDashboard extends JFrame {
12:     int userId,patientId; DefaultTableModel model=new DefaultTableModel(); JTable table=new JTable(model);
13:     public PatientDashboard(int uid){
14:         userId=uid;
15:         setTitle("Patient Dashboard");setSize(1000,600);setLocationRelativeTo(null);setDefaultCloseOperation(EXIT_ON_CLOSE);UI.style(this);
16:         try{patientId=new PatientDAO().findIdByUser(uid);}catch(Exception e){UI.error(this,e);}
17:         JPanel bar=new JPanel();JButton d=UI.button("Doctors"),b=UI.button("Book Appointment"),a=UI.button("My Appointments"),r=UI.button("Medical Records"),out=UI.butt
18:         for(JButton x:new JButton[]{d,b,a,r,out})bar.add(x);add(bar,BorderLayout.NORTH);add(new JScrollPane(table));
19:         d.addActionListener(e->doctors());b.addActionListener(e->book());a.addActionListener(e->appointments());r.addActionListener(e->records());out.addActionListener(e->
20:     }
21:     void doctors(){try{model.setDataVector(new Object[]{"ID","Doctor","Specialization","Phone","Email","Available","Fee"},0);for(Doctor x:new DoctorDAO().all())model.addR
22:     void book(){
23:         try{
24:             String d=JOptionPane.showInputDialog(this,"Doctor ID:");String date=JOptionPane.showInputDialog(this,"Date (YYYY-MM-DD):");
25:             String time=JOptionPane.showInputDialog(this,"Time (HH:MM:SS):");String reason=JOptionPane.showInputDialog(this,"Reason:");
26:             if(d==null||date==null||time==null)return;
27:             new AppointmentDAO().add(new Appointment(patientId,Integer.parseInt(d),Date.valueOf(date),Time.valueOf(time),reason));
28:             JOptionPane.showMessageDialog(this,"Appointment booked successfully.");appointments();
29:         }catch(Exception e){UI.error(this,e);}
30:     }
31:     void appointments(){try{model.setDataVector(new Object[]{"ID","Patient","Doctor","Date","Time","Reason","Status"},0);for(String[] x:new AppointmentDAO().all())if(x[1]
32:     void records(){try{model.setDataVector(new Object[]{"ID","Patient","Doctor","Diagnosis","Prescription","Notes","Date"},0);for(String[] x:new MedicalRecordDAO().all())if(
33: }
```

src/com/healthcare/gui/DoctorDashboard.java

```
 1: package com.healthcare.gui;
 2:
 3: import com.healthcare.dao.*;
 4: import com.healthcare.model.MedicalRecord;
 5: import javax.swing.*;
 6: import javax.swing.table.DefaultTableModel;
 7: import java.awt.*;
 8: import java.sql.Date;
 9:
10: public class DoctorDashboard extends JFrame {
11:     int userId;DefaultTableModel model=new DefaultTableModel();JTable table=new JTable(model);
12:     public DoctorDashboard(int uid){
13:         userId=uid;setTitle("Doctor Dashboard");setSize(1050,600);setLocationRelativeTo(null);setDefaultCloseOperation(EXIT_ON_CLOSE);UI.style(this);
14:         JPanel bar=new JPanel();JButton a=UI.button("Appointments"),ok=UI.button("Approve"),no=UI.button("Reject"),done=UI.button("Complete"),rec=UI.button("Add Medica
15:         for(JButton x:new JButton[]{a,ok,no,done,rec,out})bar.add(x);add(bar,BorderLayout.NORTH);add(new JScrollPane(table));
16:         a.addActionListener(e->appointments());ok.addActionListener(e->status("Approved"));no.addActionListener(e->status("Rejected"));done.addActionListener(e->status("
17:     }
18:     void appointments(){try{model.setDataVector(new Object[]{"ID","Patient","Doctor","Date","Time","Reason","Status"},0);for(String[] x:new AppointmentDAO().all())model
19:     void status(String s){
20:         int row=table.getSelectedRow();if(row<0){JOptionPane.showMessageDialog(this,"Select appointment.");return;}
21:         try{new AppointmentDAO().status(Integer.parseInt(model.getValueAt(row,0).toString()),s);appointments();}catch(Exception e){UI.error(this,e);}
22:     }
23:     void record(){
24:         try{
25:             String p=JOptionPane.showInputDialog(this,"Patient ID:");String d=JOptionPane.showInputDialog(this,"Doctor ID:");
26:             String diag=JOptionPane.showInputDialog(this,"Diagnosis:");String pre=JOptionPane.showInputDialog(this,"Prescription:");
27:             String notes=JOptionPane.showInputDialog(this,"Notes:");
28:             if(p==null||d==null||diag==null)return;
29:             new MedicalRecordDAO().add(new MedicalRecord(Integer.parseInt(p),Integer.parseInt(d),diag,pre,notes,new Date(System.currentTimeMillis())));
30:             JOptionPane.showMessageDialog(this,"Medical record saved.");
31:         }catch(Exception e){UI.error(this,e);}
32:     }
33: }
```

README.md

```
# Online Health Care Management System

## Academic Project
Java Swing + JDBC + MySQL healthcare management system.

## Major Features
- Role-based authentication
- Admin dashboard
- Doctor dashboard
- Patient dashboard
- Doctor management
- Patient management
- Appointment booking
- Appointment status workflow
- Medical records
- Consultation fee
- MySQL database
- JDBC PreparedStatements
- DAO architecture
- OOP, inheritance and polymorphism
- Interface and generics
- Collections
- Multithreading and synchronization
- Exception handling

## Architecture
GUI -> Service -> DAO -> JDBC -> MySQL

## Requirements
- JDK 8+
- MySQL Server
- MySQL Connector/J
- IntelliJ IDEA / Eclipse / NetBeans

## Setup
1. Create/open the project.
2. Run `database/healthcare.sql` in MySQL.
3. Change MySQL username/password in `DBConnection.java`.
4. Add MySQL Connector/J.
5. Run `com.healthcare.Main`.

## Demo Accounts
Admin: admin / admin123
Doctor: doctor1 / doctor123
Patient: patient1 / patient123

## GitHub Submission
Upload the complete project, SQL file, README and screenshots.
Make the repository public or give reviewer access.

## Important
Credentials are demonstration values only. A real application should hash passwords and use secure configuration.
```

pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.healthcare</groupId><artifactId>online-healthcare-management-system</artifactId><version>2.0</version>
 <properties><maven.compiler.source>8</maven.compiler.source><maven.compiler.target>8</maven.compiler.target></properties>
 <dependencies>
  <dependency><groupId>com.mysql</groupId><artifactId>mysql-connector-j</artifactId><version>9.4.0</version></dependency>
 </dependencies>
</project>
```

3. Rubric Coverage

OOP: User inheritance hierarchy, overridden dashboard methods, encapsulated model fields, interface-based CRUD design. Collections & Generics: List/ArrayList and CRUDOperations. Exception Handling: SQL and input exceptions plus custom ValidationException. Multithreading: NotificationService extends Thread. Synchronization: AppointmentCounter uses synchronized methods. Database Operations: DAO classes. JDBC: DBConnection, PreparedStatement and ResultSet. GUI: Swing frames and dashboards.

4. Setup

1. Install JDK 8+ and MySQL.
2. Run database/healthcare.sql in MySQL.
3. Set USER and PASSWORD in DBConnection.java.
4. Add MySQL Connector/J (pom.xml is provided for Maven).
5. Run com.healthcare.Main.
6. Use the demo credentials listed in README.md.

5. Submission Checklist

[ ] Test login for all roles
[ ] Test appointment booking
[ ] Test approve/reject/complete
[ ] Test medical records
[ ] Add actual screenshots
[ ] Upload complete project to GitHub
[ ] Keep README.md in repository
[ ] Include database/healthcare.sql
[ ] Submit GitHub repository link
[ ] Submit presentation/PPT with screenshots and diagrams